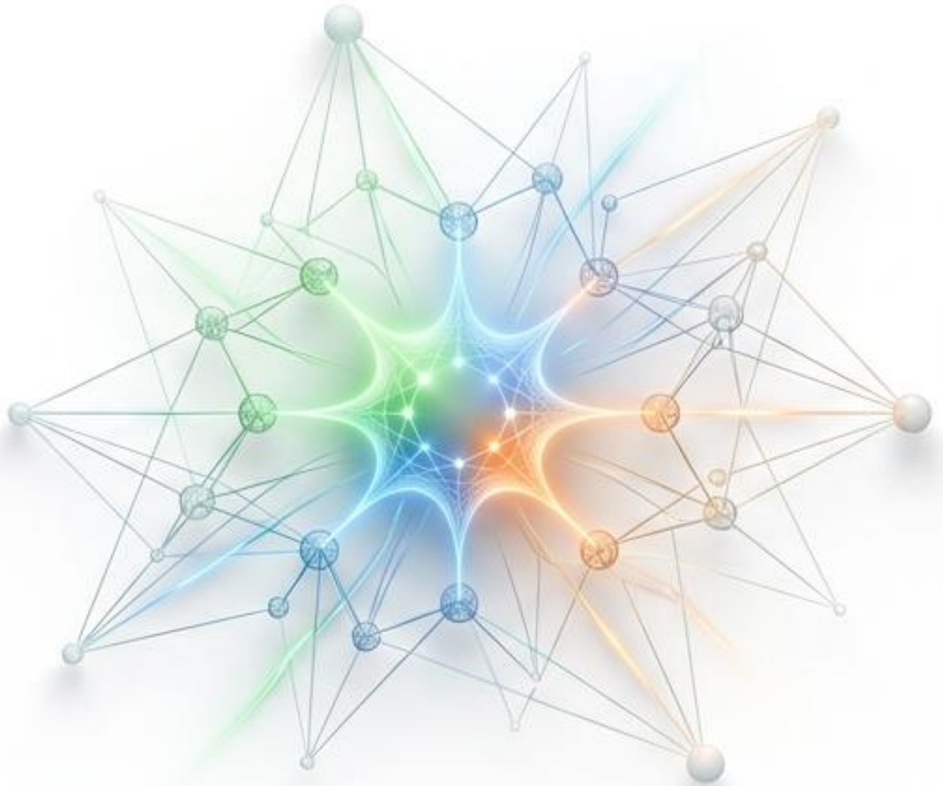


Chương 10: Giới thiệu về Mạng Nơ-ron Nhân tạo

Giảng viên
Đại học Đông Á
August 11, 2026

Gemini Notebook

Mục tiêu của Chương



Khái niệm nền tảng về Nơ-ron sinh học và Nơ-ron nhân tạo.



Kiến trúc Perceptron và Mạng Perceptron Đa lớp (MLP).



Hiểu nguyên lý hoạt động của thuật toán Lan truyền ngược (Backpropagation).



Triển khai mạng MLP cho bài toán Hồi quy và Phân loại.



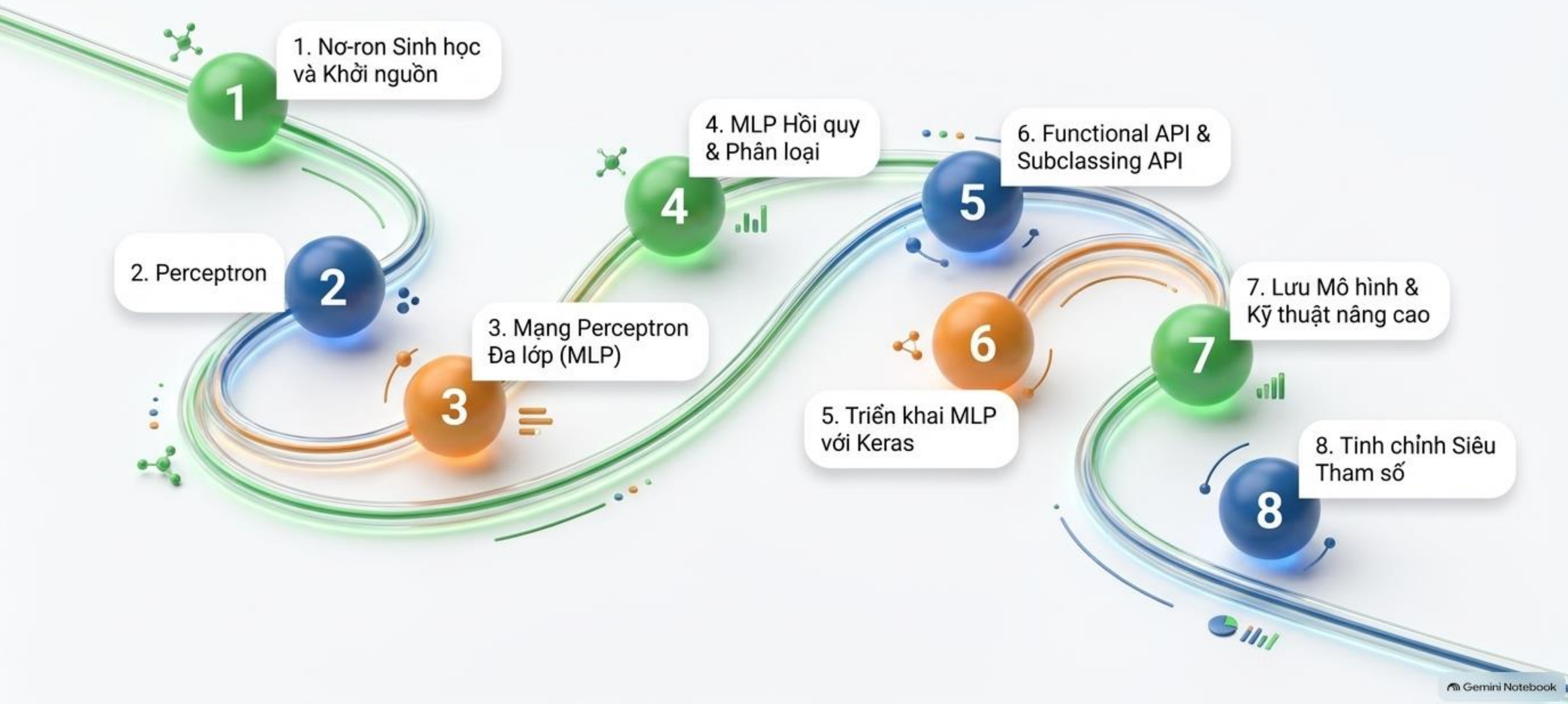
Sử dụng Keras và TensorFlow để xây dựng mô hình. Khám phá các API: Sequential, Functional, Subclassing.



Callbacks, TensorBoard và tinh chỉnh siêu tham số.

Gemini Notebook

Mục lục



Gemini Notebook

1.1 Lịch sử Mạng Nơ-ron Nhân tạo (ANN)



1943

Khởi nguồn: Ra đời bởi Warren McCulloch và Walter Pitts. Truyền cảm hứng từ cấu trúc bộ não con người.



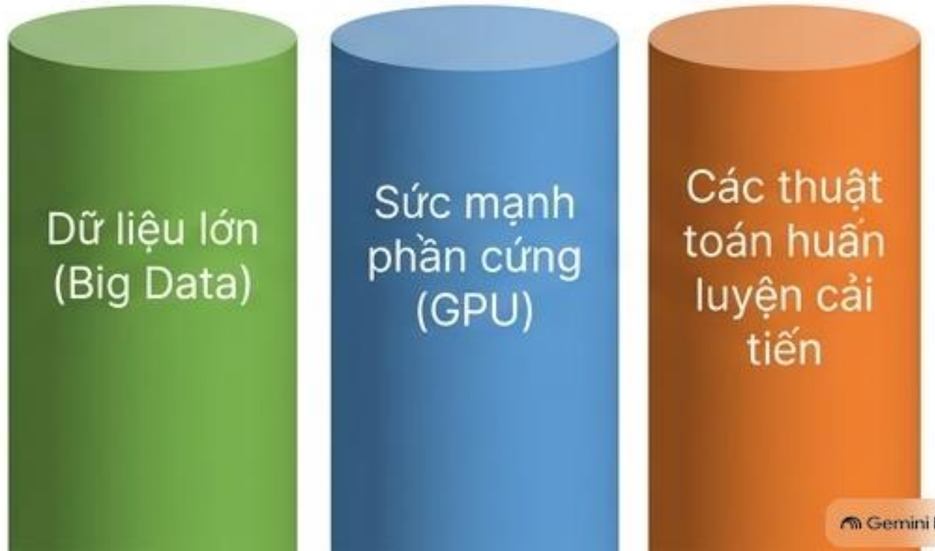
Mid Century

Thăng trầm: Trải qua nhiều thập kỷ chững lại (AI Winter).



2010s

Bùng nổ: Mạng nơ-ron bùng nổ trở lại.

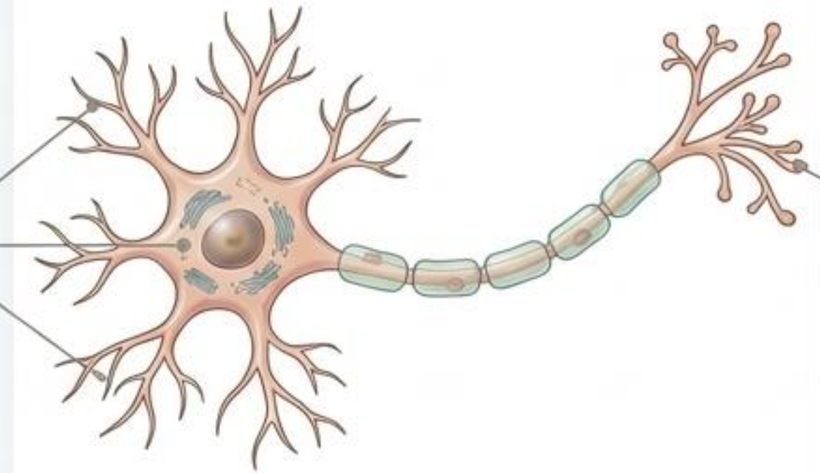


Gemini Notebook

1.2 Nơ-ron Sinh học

Tế bào thần kinh sinh học là một tế bào dị thường nằm ở vỏ não.

Cấu trúc: Thân tế bào, sợi nhánh (Dendrites), và sợi trục (Axon).

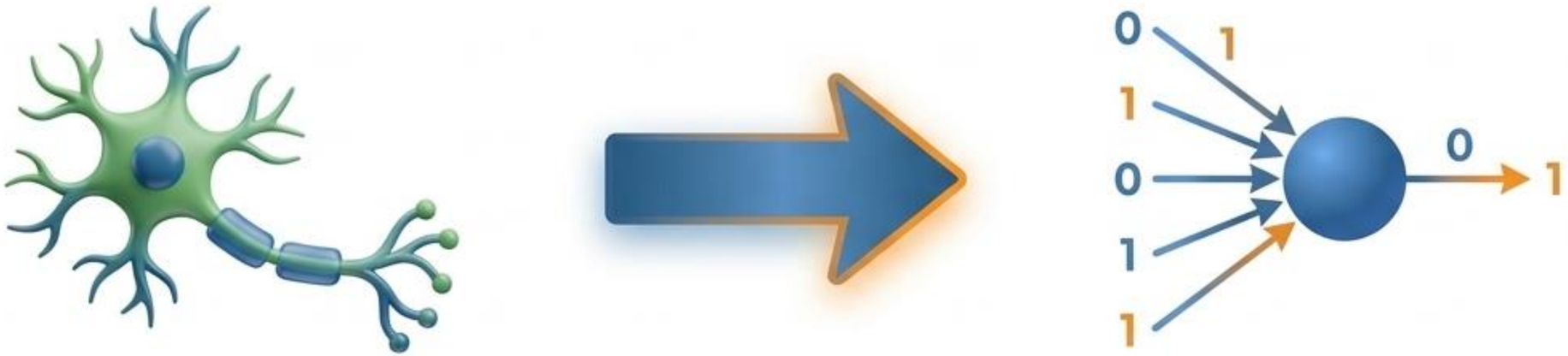


Tín hiệu được truyền qua các khớp thần kinh (Synapses).

Hình 10-1: Cấu trúc của Nơ-ron sinh học

Gemini Notebook

1.3 Từ Sinh học đến Mô hình Nhân tạo



Mô hình nơ-ron nhân tạo đầu tiên (Mô hình McCulloch-Pitts) cực kỳ đơn giản.

Nhận một hoặc nhiều đầu vào (nhị phân), và tạo ra một đầu ra.

Nơ-ron kích hoạt (phát tín hiệu 1) khi một lượng đủ lớn các đầu vào của nó kích hoạt.

Khi kết nối thành mạng lưới, có thể tính toán bất kỳ biểu thức logic nào.

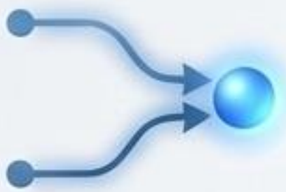
Gemini Notebook

1.4 Các Phép toán Logic với Nơ-ron

Mạng (a): Hàm IDENTITY.

Đầu ra phản ánh trực tiếp đầu vào mà không có sự thay đổi.



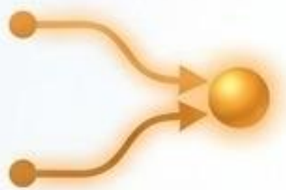


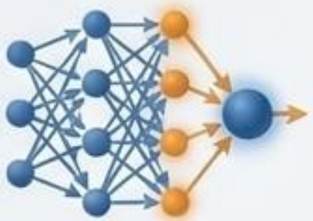
Mạng (b): Hàm AND (Chỉ sáng khi cả hai đều sáng).

Cổng chỉ kích hoạt khi tất cả các tín hiệu đầu vào đều ở mức cao (1).

Mạng (c): Hàm OR (Sáng khi 1 trong 2 sáng).

Kích hoạt nếu ít nhất một tín hiệu đầu vào ở mức cao (1).





Mạng (d): Cổng logic phức tạp.

Một mạng nơ-ron đa lớp có thể thực hiện các phép toán phi tuyến tính phức tạp hơn.

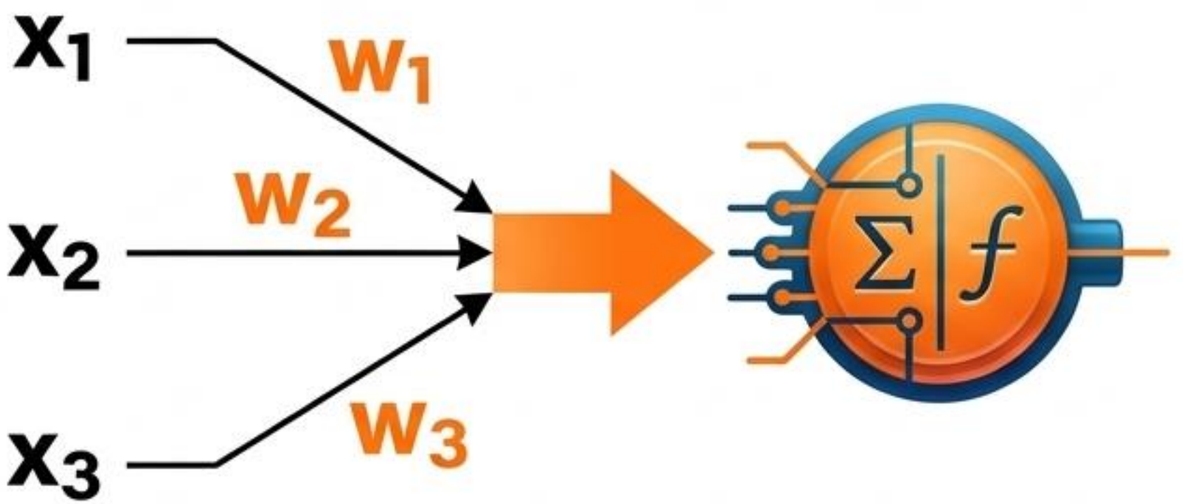
Hình 10-2: Các phép tính logic đơn giản

Gemini Notebook

2.1 Khái niệm Perceptron

1957

Phát minh năm 1957 bởi Frank Rosenblatt.



Dựa trên TLU (Threshold Logic Unit) hay LTU (Linear Threshold Unit).

Thay vì nhị phân, TLU nhận đầu vào là các con số.

Mỗi kết nối đầu vào được gán một trọng số (Weight) **w**.

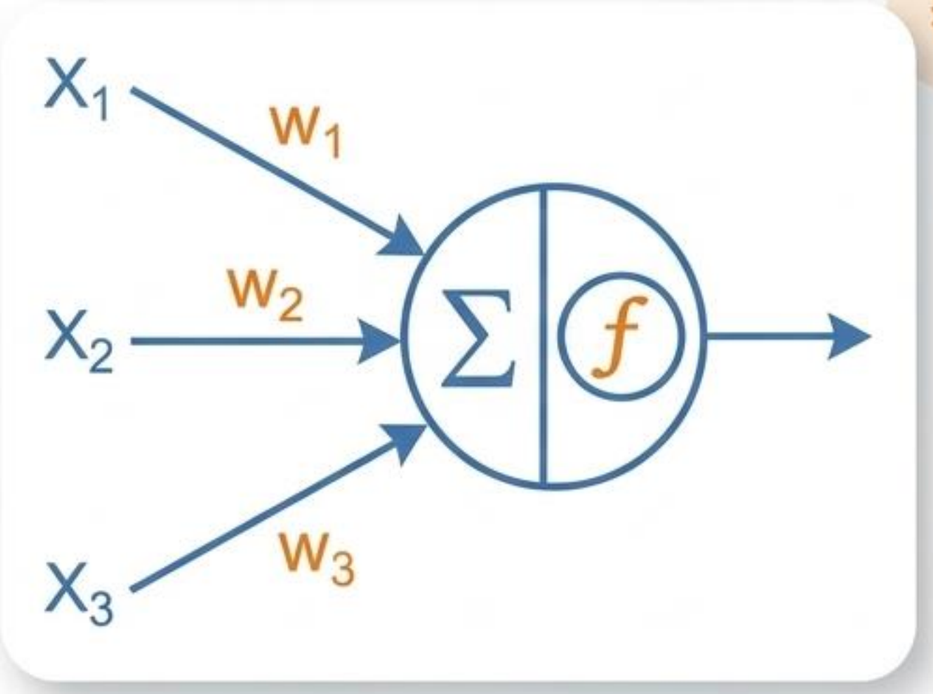
Gemini Notebook

2.2 Hoạt động của TLU (Threshold Logic Unit)

1. Tính tổng trọng số:
$z = w_1x_1 + w_2x_2 + ...+ b = w^Tx + b.$

2. Áp dụng hàm bước (Step function) vào z: $h_w(x) = step(z)$.

Hàm bước thường dùng là hàm Heaviside (Trả về 1 nếu $z \geq 0$, ngược lại 0).

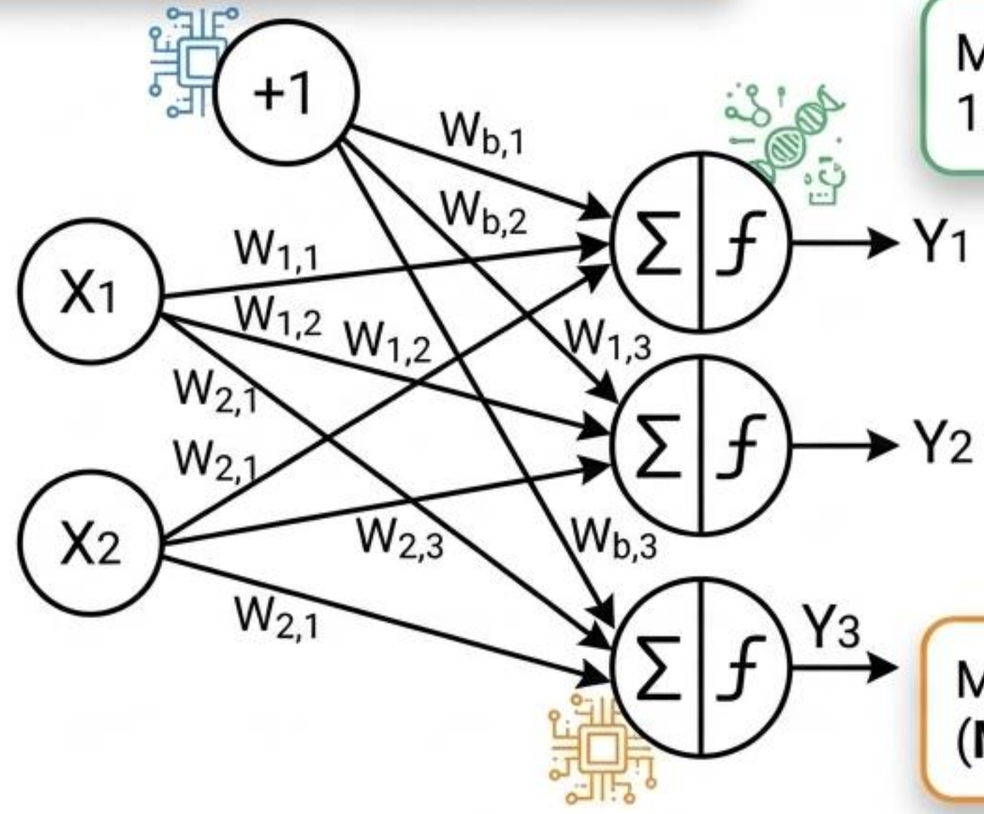


Hình 10-3: *Threshold Logic Unit (TLU)*

Gemini Notebook

2.3 Kiến trúc Mạng Perceptron

Bao gồm một lớp các TLU kết nối với tất cả các đầu vào (Lớp dày đặc - Dense Layer).



Mạng trong hình có 2 đầu vào, 1 nơ-ron bias, và 3 nơ-ron đầu ra.

Mạng này có thể phân loại đa nhãn (**Multi-output classification**).

Hình 10-4: Mạng Perceptron

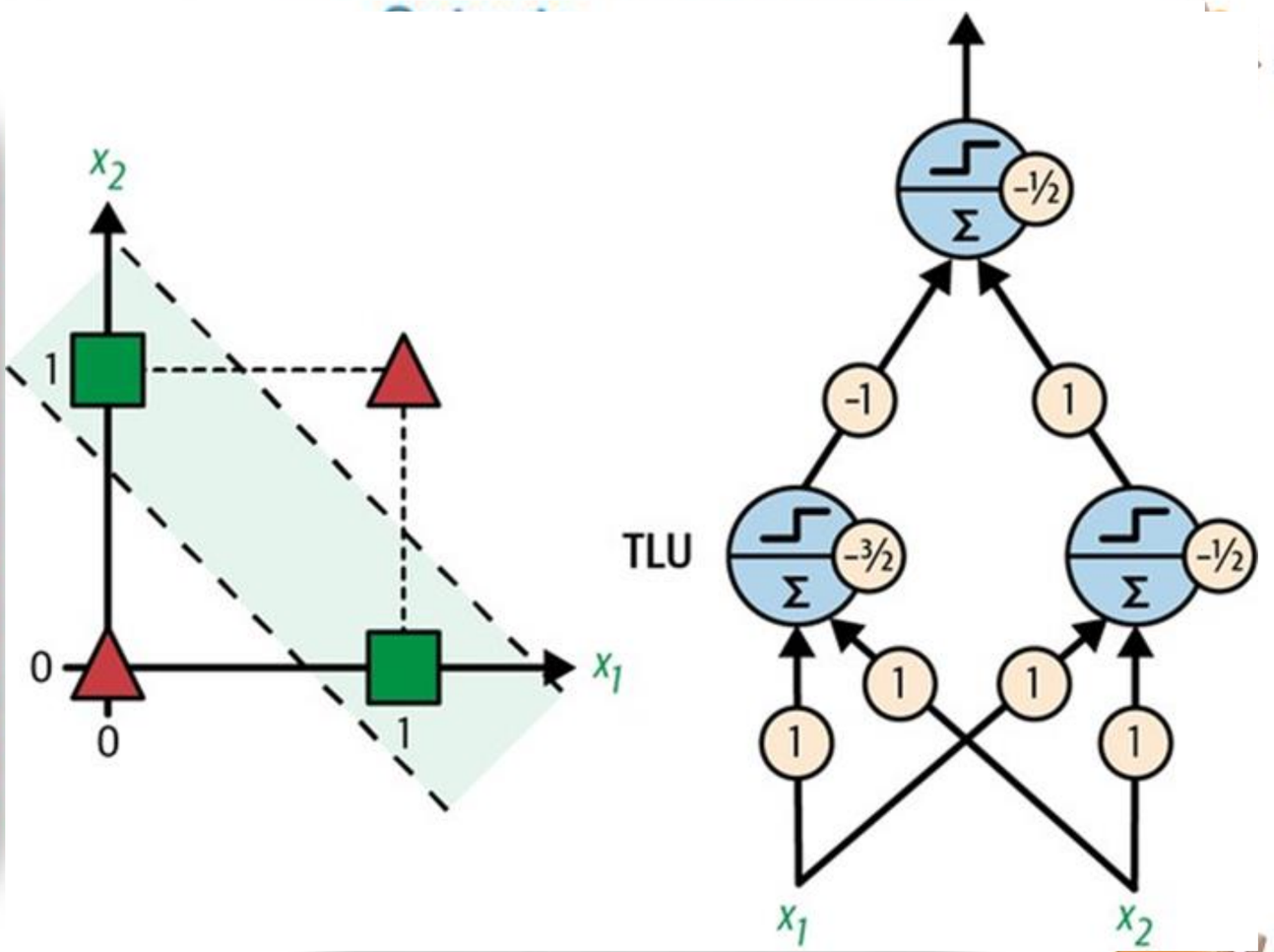
Gemini Notebook

2.4 Điểm yếu của Perceptron: Bài toán XOR

Năm 1969, Minsky và Papert chứng minh Perceptron có nhược điểm nghiêm trọng.

Chúng là các bộ phân loại tuyến tính, do đó không thể giải bài toán XOR (Exclusive OR).

Hình bên cho thấy bài toán XOR không thể bị cắt đôi bằng một đường thẳng.

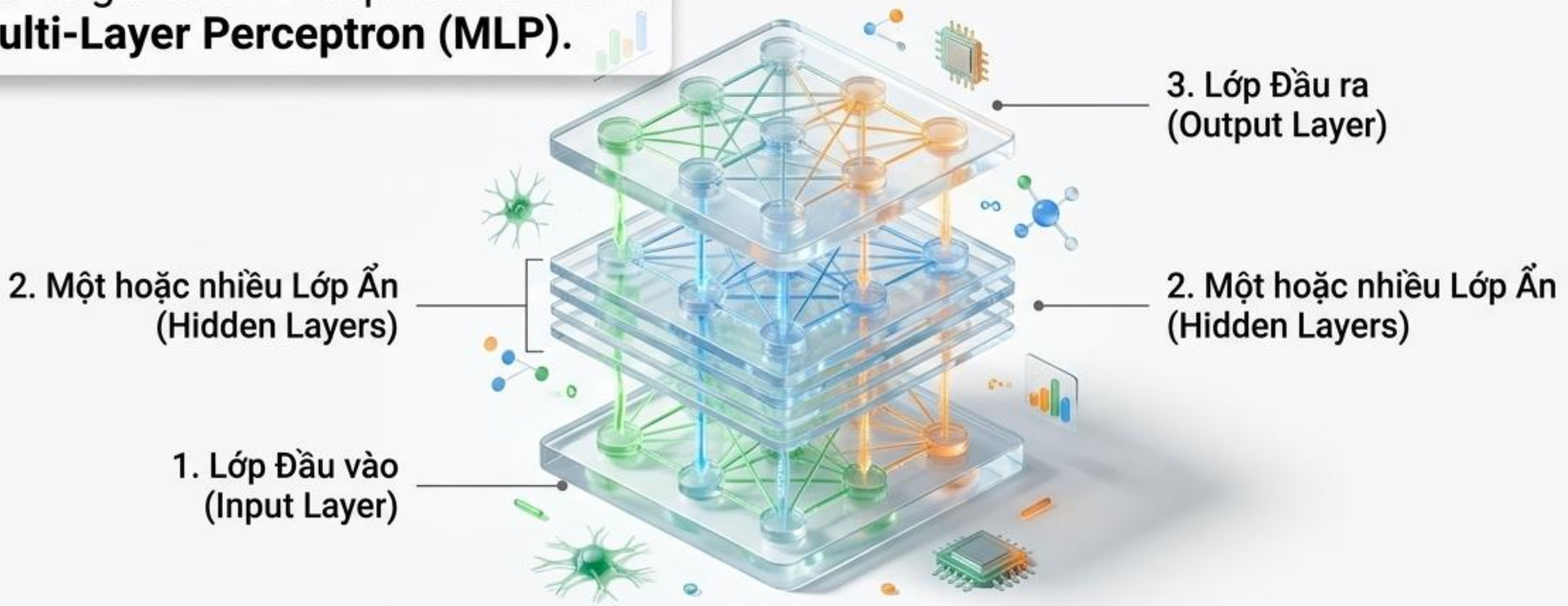


Hình 10-5: Điểm yếu giải quyết XOR

Gemini Notebook

3.1 Mạng Perceptron Đa Lớp (MLP)

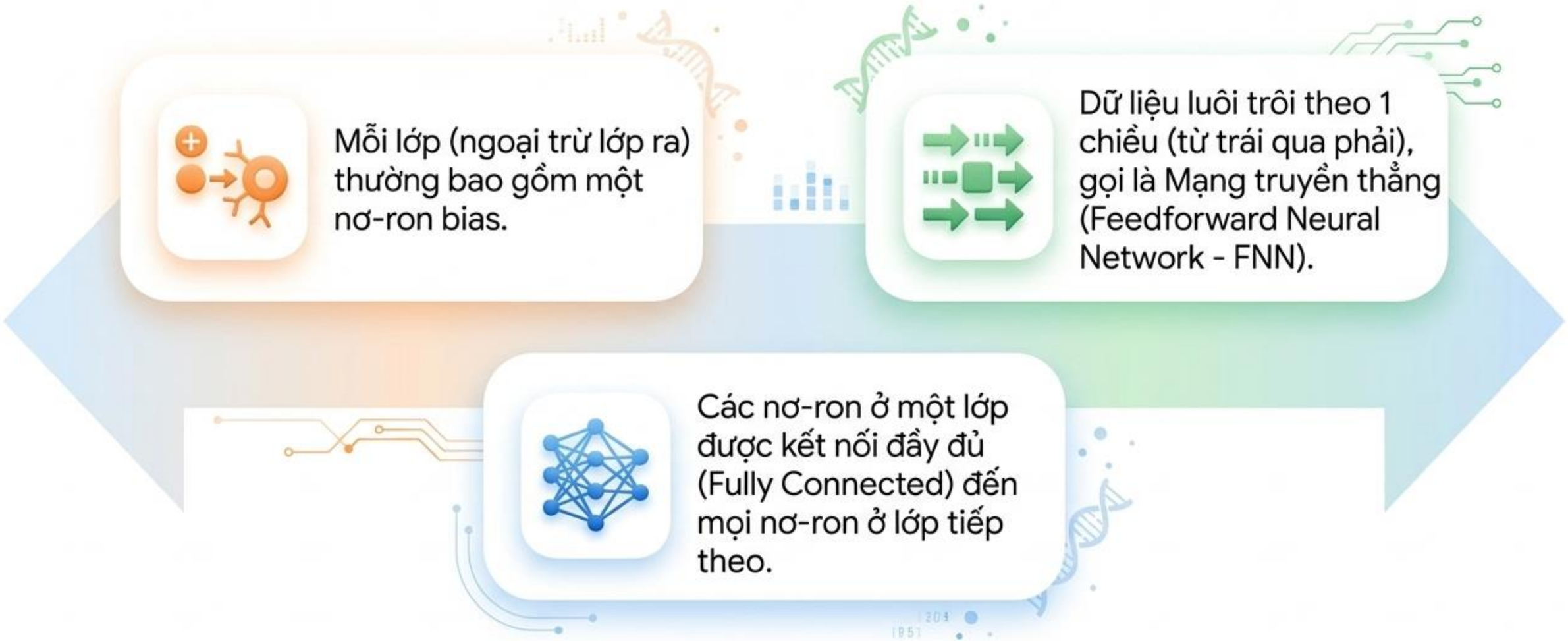
Khắc phục sự cố XOR bằng cách xếp chồng nhiều Perceptron lên nhau → **Multi-Layer Perceptron (MLP).**



Bất kỳ mạng nơ-ron nào có từ 2 lớp ẩn trở lên được gọi là Học Sâu (Deep Learning).

Gemini Notebook

3.2 Kiến trúc MLP

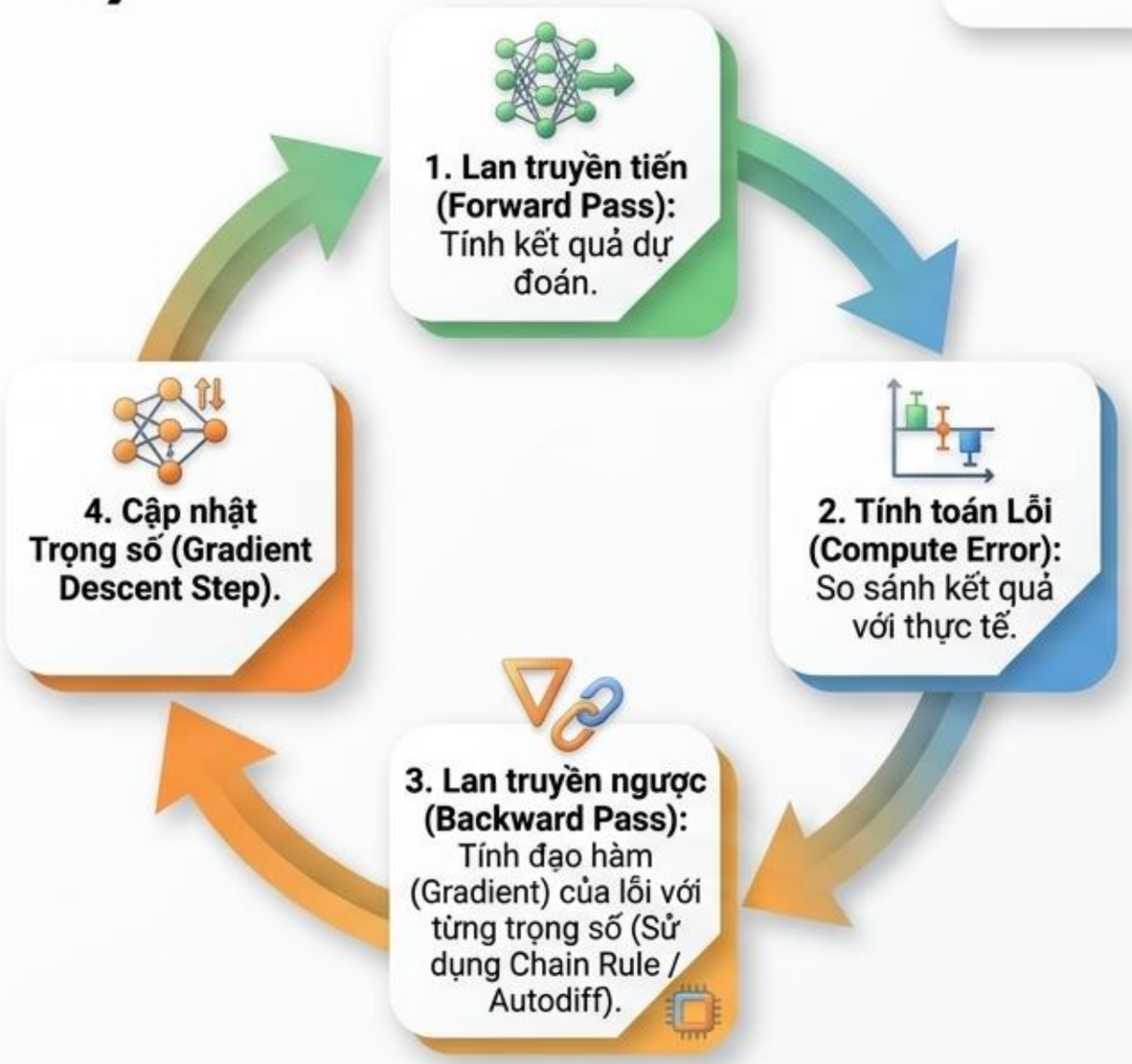


Hình 10-6: Kiến trúc Mạng Perceptron Đa lớp

Gemini Notebook

3.3 Thuật toán Lan truyền ngược (Backpropagation)

1986 - Rumelhart, Hinton, Williams công bố Backpropagation kết hợp Gradient Descent. Linh hồn của Deep Learning.



Gemini Notebook

3.4 Tại sao lại cần Hàm Kích hoạt (Activation Functions)?

Để Backprop hoạt động, thay hàm bước (Step function) bằng các hàm liên tục có đạo hàm khác không.

Vấn đề (Tuyến tính)



Nếu chỉ cộng dồn tuyến tính, mạng nơ-ron 100 lớp cũng chỉ ngang bằng mạng 1 lớp (Vì hàm tuyến tính tổ hợp lại vẫn là tuyến tính).

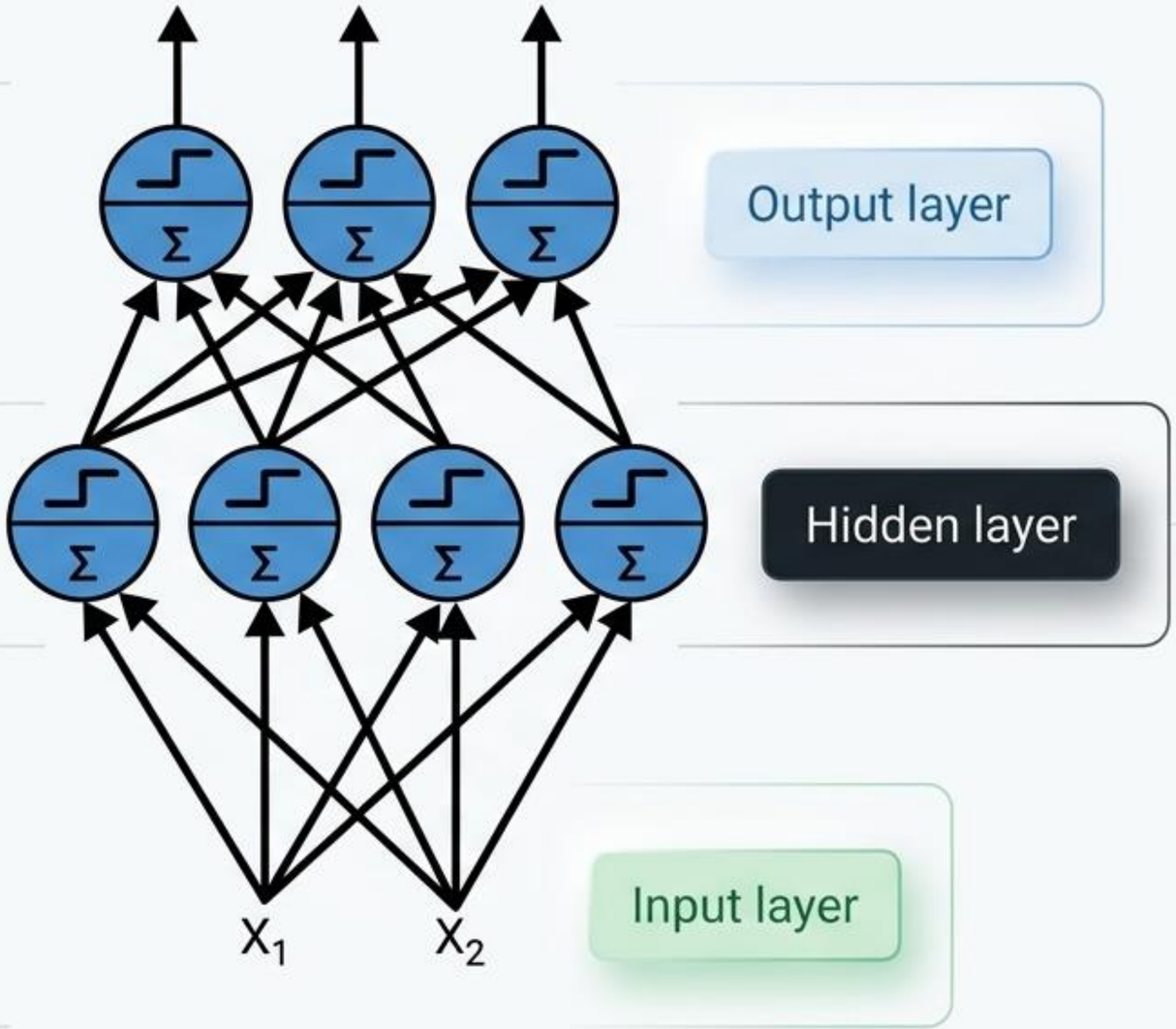
Giải pháp (Phi tuyến tính)



Hàm kích hoạt mang tính phi tuyến tính (Non-linearity), giúp mạng học được các hàm phức tạp.

Gemini Notebook

Kiến trúc Mạng Nơ-ron Cơ bản

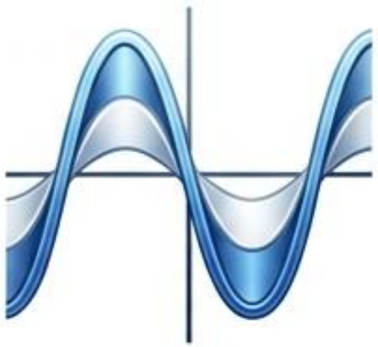


Gemini Notebook

Các Hàm Kích Hoạt Phổ Biến



Sigmoid: Ép đầu ra về khoảng [0, 1]. Dùng cho phân loại nhị phân.



Tanh: Ép về [-1, 1], có tâm tại 0.



ReLU: Bằng 0 nếu âm, giữ nguyên nếu dương. Nhanh, đạo hàm tốt, là mặc định cho các lớp ẩn hiện nay.

Hình 10-7: Bảng các Hàm kích hoạt (Activation Functions)

Gemini Notebook

MLP Hồi quy (Regression MLP)



Hồi quy dự đoán một giá trị liên tục (Ví dụ: Giá nhà).

Lớp đầu ra chỉ cần **1 nơ-ron duy nhất** (Không dùng hàm kích hoạt).

Nếu dự đoán giá trị phải luôn dương, dùng hàm kích hoạt **ReLU** hoặc **Softplus** ở đầu ra.

Hàm mất mát (Loss function): **MSE (Mean Squared Error)** hoặc **MAE (Mean Absolute Error)**.

Gemini Notebook

Cấu hình chuẩn cho MLP Hồi quy

Hyperparameter	Typical Value
Nơ-ron đầu vào (Input)	Một nơ-ron cho mỗi đặc trưng (Feature).
Số lớp ẩn (Hidden Layers)	Tùy thuộc bài toán, thường từ 1 đến 5.
Nơ-ron mỗi lớp ẩn	Thường từ 10 đến 100.
Nơ-ron đầu ra (Output)	1 (Nếu tiên đoán 1 giá trị), N (Nếu tiên đoán N chiều).
Hàm kích hoạt (Lớp ẩn)	**ReLU** (Hoặc **SELU**, **Mish**).
Hàm kích hoạt (Đầu ra)	**None** (Liên tục), **ReLU** (Dương), **Sigmoid** (Khoảng 0-1).
Hàm mất mát (Loss)	**MSE**, **MAE**, hoặc **Huber Loss**.

Hình 10-8: Kiến trúc MLP Hồi quy chuẩn

Gemini Notebook

MLP Phân loại (Classification MLP)



Phân loại nhị phân (Binary): Dùng 1 nơ-ron đầu ra với hàm **Sigmoid**. (Xác suất từ 0 đến 1). Loss: Binary Crossentropy.



Phân loại đa nhãn (Multilabel): Vd nhận dạng email vừa là spam **vừa là urgent.** Dùng N nơ-ron đầu ra, mỗi nơ-ron có hàm **Sigmoid**. Loss: Binary Crossentropy.



Phân loại đa lớp (Multiclass): Vd nhận diện MNIST (Từ 0 đến 9). Dùng N nơ-ron đầu ra với hàm **Softmax**. Loss: Categorical Crossentropy.

Gemini Notebook

Cấu hình chuẩn cho MLP Phân loại

Hyperparameter	Binary	Multilabel	Multiclass
Input & Hidden	Giống như Hồi quy (1-5 lớp, ReLU)		
Output Neurons	1	1 cho mỗi nhãn	1 cho mỗi lớp (Tổng cộng N)
Output Activation	**Sigmoid**	**Sigmoid**	**Softmax**
Loss Function	Binary Crossentropy	Binary Crossentropy	Categorical Crossentropy

Hình 10-9: Kiến trúc MLP Phân loại chuẩn

Gemini Notebook

TensorFlow và Keras



TensorFlow: Thư viện mã nguồn mở cho Học sâu của Google. Xử lý đồ thị tính toán mạnh mẽ trên CPU, GPU, TPU.



Keras: API bậc cao (High-level API) của TensorFlow. Cực kỳ thân thiện với người dùng, dễ học, nhưng đủ linh hoạt để tuỳ chỉnh mô hình chuyên sâu.

Hướng dẫn sau sử dụng bộ dữ liệu **Fashion MNIST** (Nhận diện ảnh 28x28 của các mặt hàng quần áo thời trang).

Gemini Notebook

Sequential API: Xây dựng mô hình

Keras **Sequential API** tạo mô hình bằng cách xếp chồng các lớp (Layer) tuần tự.

```
import tensorflow as tf
from tensorflow import keras

model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
model.summary() # Xem tom tat cau truc mo hinh
```

Gemini Notebook

Giải thích Mô hình Sequential



Flatten: Lớp chuyển đổi ảnh ma trận 2D (28x28) thành mảng 1D (784). (Không có trọng số).



Dense(300): Lớp ẩn đầu tiên với 300 nơ-ron kết nối đầy đủ. Số tham số = 784 x 300 + 300 (bias) = 235,500.



Dense(100): Lớp ẩn thứ hai với 100 nơ-ron. (30,100 tham số).



Dense(10): Lớp đầu ra với 10 nơ-ron (cho 10 lớp mặt hàng). Dùng Softmax để xuất ra xác suất tổng bằng 1.

Gemini Notebook

Biên dịch (Compile) và Huấn luyện (Fit)

```
# Buóc 1: Compile - Khai bao Loss va Optimizer
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd", metrics=["accuracy"])

# Buóc 2: Fit - Huấn luyện mo hình (Lẫn truyện nguốc)
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

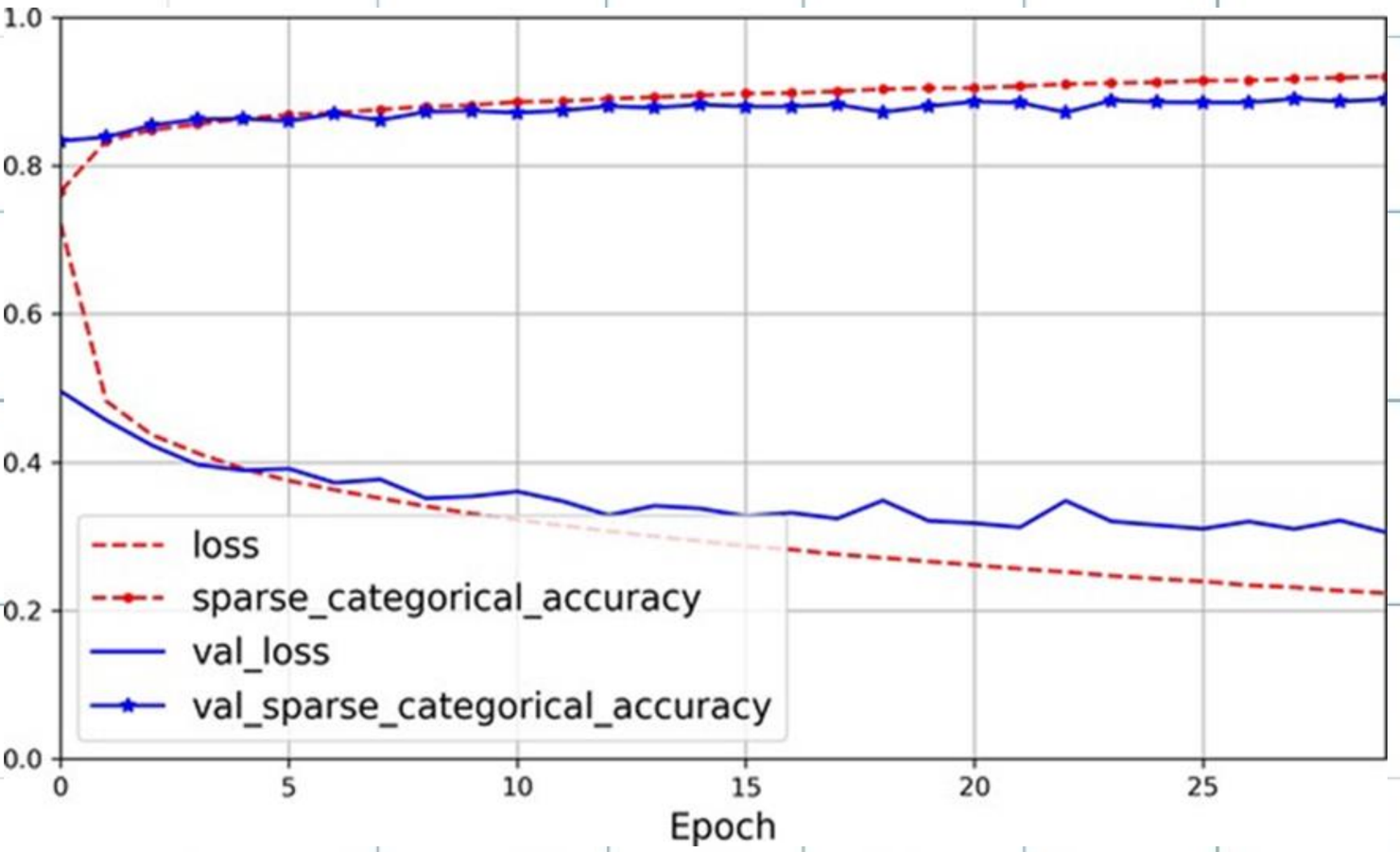
Dùng vì nhãn Y là các số nguyên rải rác (0-9).

Stochastic Gradient Descent.

Huấn luyện duyệt qua toàn bộ tập dữ liệu 30 lần.

Gemini Notebook

Đường cong Học tập (Learning Curves)



Hình 10-11: *Đồ thị Loss và Accuracy theo số vòng lặp (Epoch)*

- **Object history** chứa dữ liệu loss và accuracy của train/valid sau mỗi epoch.

Nhìn vào biểu đồ, Loss liên tục giảm dần (tốt).

Accuracy liên tục tăng (tốt).

Nếu Validation Loss đi ngang hoặc tăng lên, mô hình đang bị **Overfitting**.

Gemini Notebook

Vượt giới hạn với Functional API

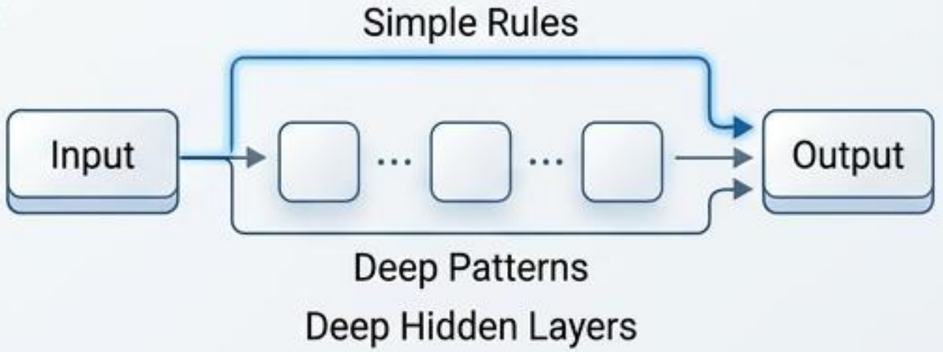
Sequential

Mô hình tuần tự (Sequential) rất phổ biến, nhưng đôi khi ta cần cấu trúc phi tuần tự. Các mô hình MLP thông thường ép mọi dữ liệu chảy qua một chuỗi các lớp ẩn, khiến các quy luật đơn giản ban đầu có thể bị biến dạng hoặc "quên" mất.



Functional

Wide & Deep Neural Network: Kiến trúc này nối trực tiếp lớp Đầu vào (Input) với lớp Đầu ra (Output) song song cùng với Lớp ẩn sâu.



Cho phép mạng học được cả **Mẫu sâu (Deep patterns)** lẫn **Quy luật nông (Simple rules)**.

Hình 10-12: Mạng Nơ-ron Rộng và Sâu (Wide & Deep)

Gemini Notebook

Kiến trúc & Code: Wide & Deep

```
input_ = keras.layers.Input(shape=X_train.shape[1:])

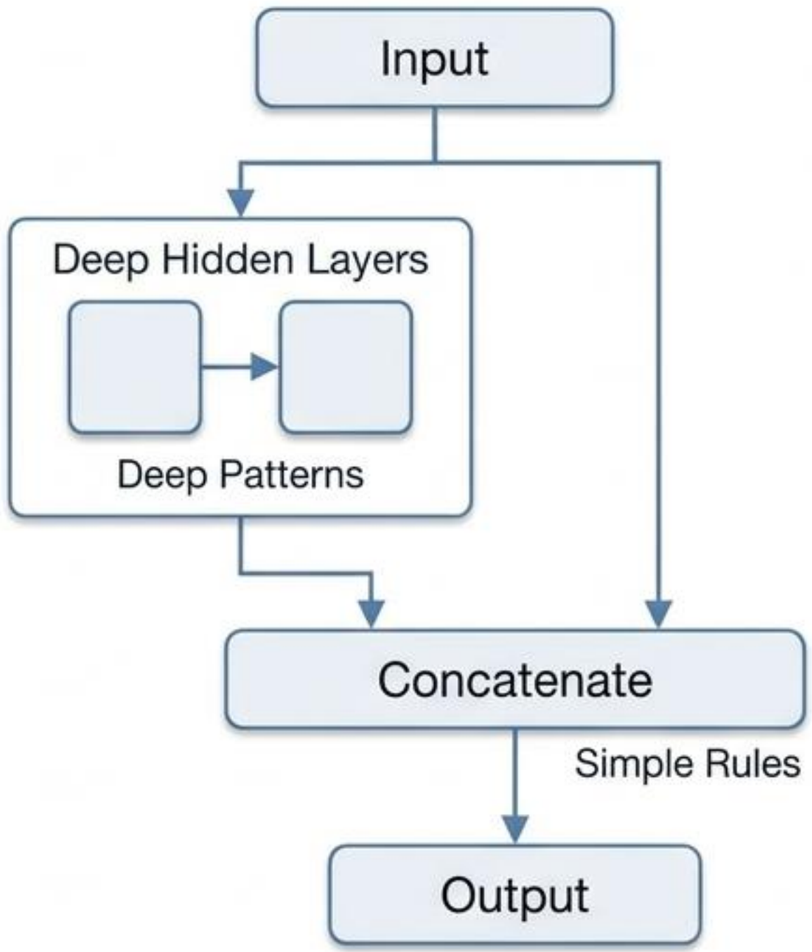
hidden1 = keras.layers.Dense(30, activation="relu")(input_)

hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)


# Ghep noi (Concatenate) duong "Wide" va "Deep"
concat = keras.layers.Concatenate()([input_, hidden2])

output = keras.layers.Dense(1)(concat)

model = keras.models.Model(inputs=[input_], outputs=[output])
```



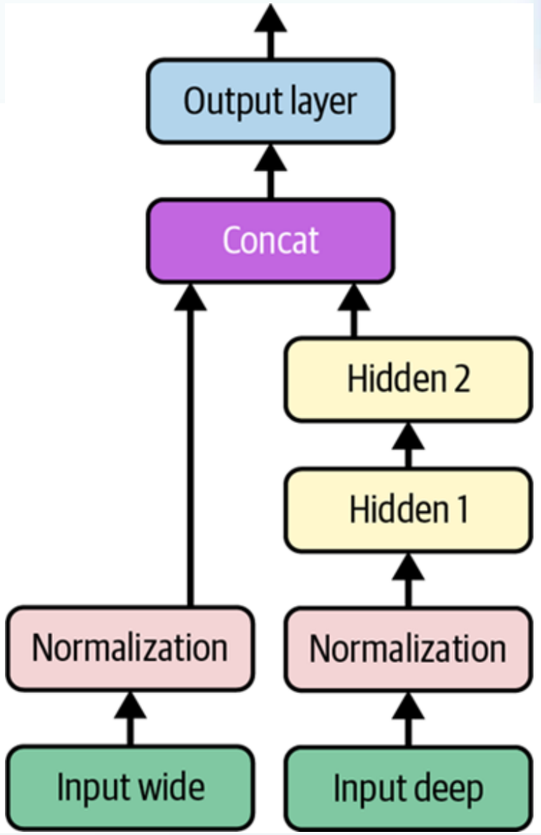
Trong Functional API, ta phải gọi từng lớp như một hàm và truyền input vào. Cuối cùng, bọc tất cả lại trong keras.models.Model.

Gemini Notebook

Functional API: Nhiều luồng dữ liệu (Multiple Inputs)

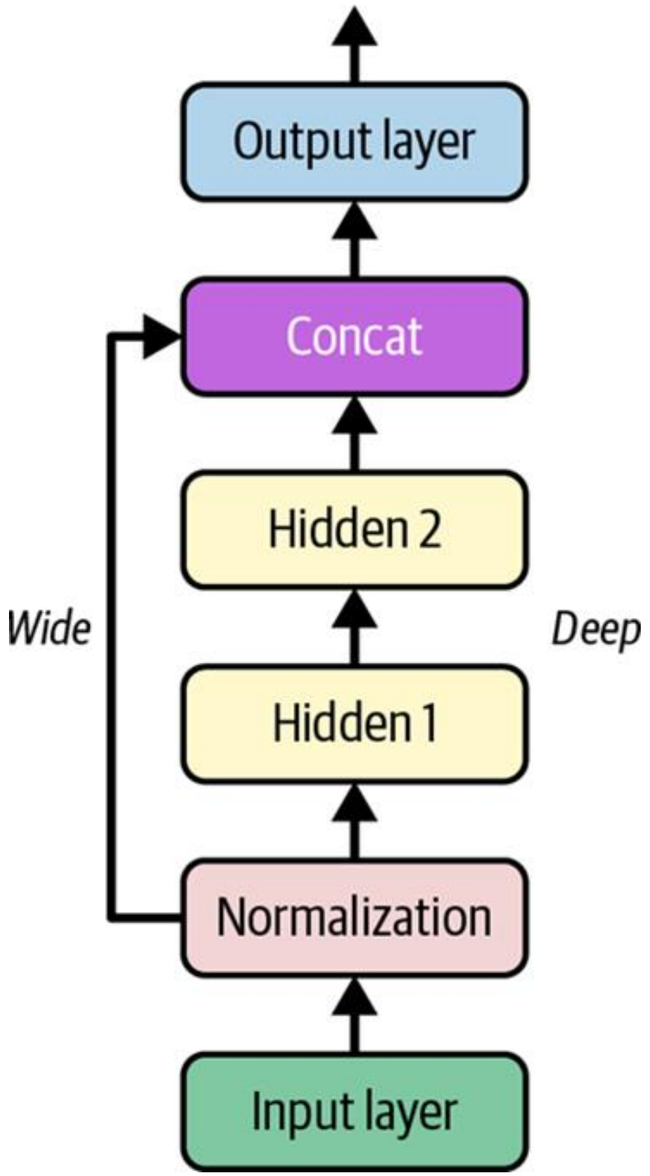
Sẽ ra sao nếu ta muốn gửi một nhóm đặc trưng vào "Đường rộng", và một nhóm đặc trưng khác vào "Đường sâu"?

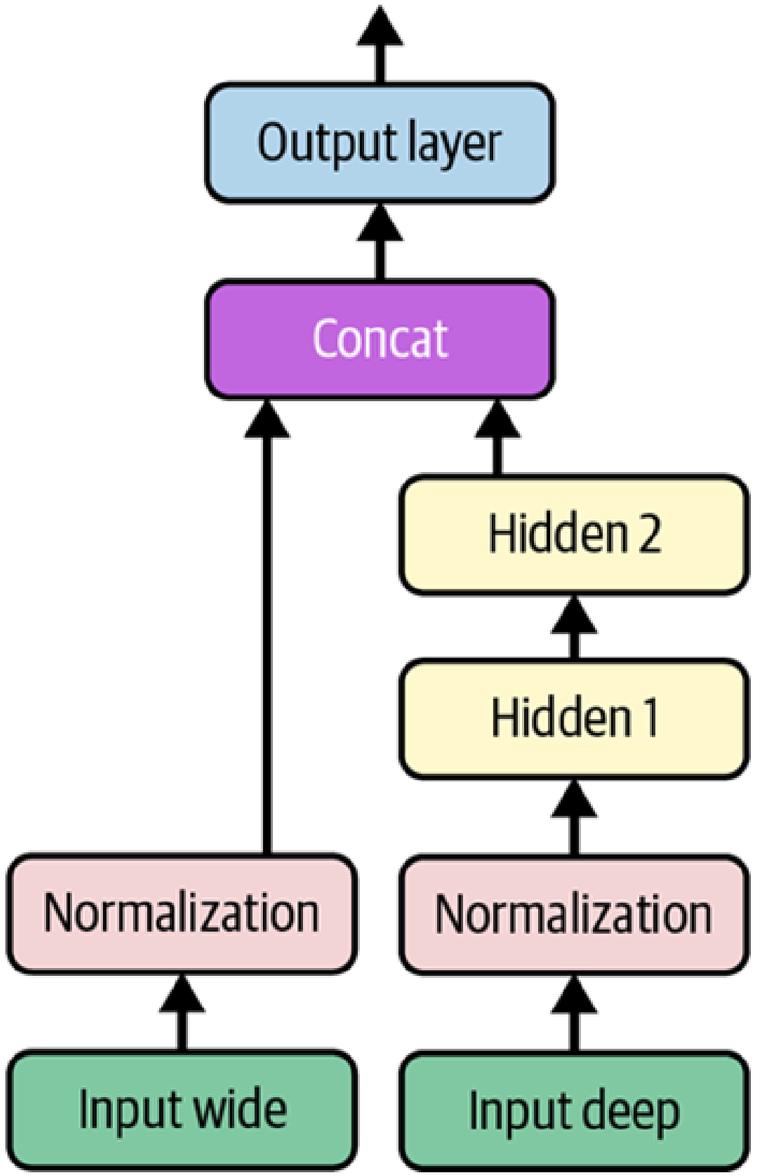
Keras Functional API có thể tạo **nhiều luồng Input**. Lúc gọi hàm fit(), sẽ truyền (**X_train_A, X_train_B**).



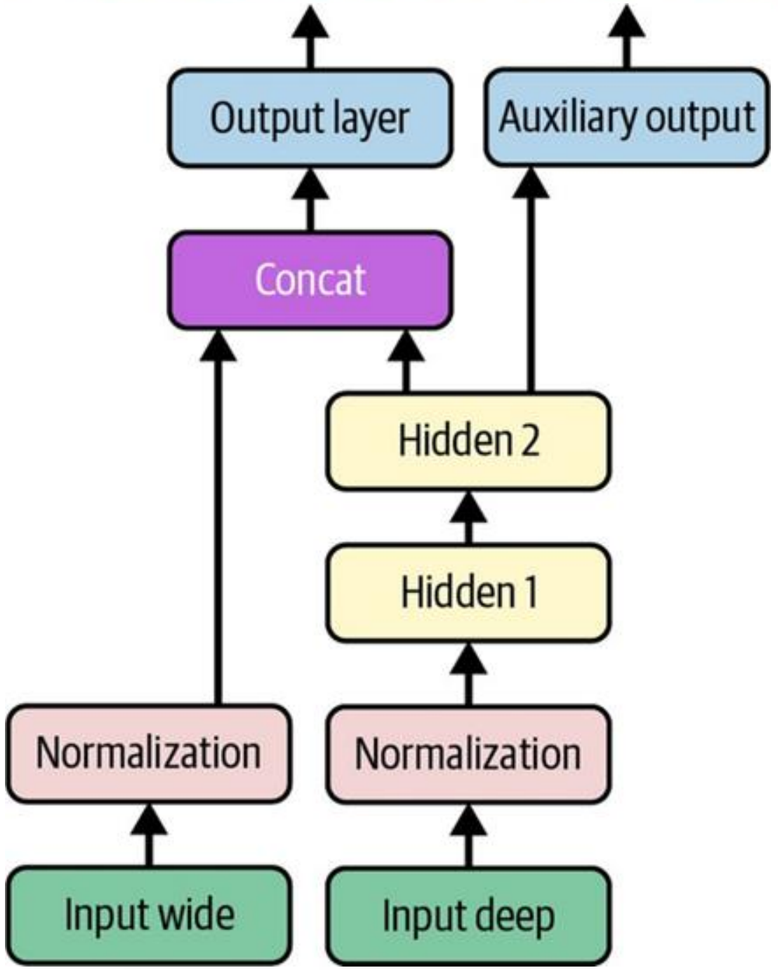
Hình 10-14: Xử lý nhiều luồng đầu vào

Gemini Notebook


Output layer
Concat
Hidden 2
Wide
Deep
Hidden 1
Normalization
Input layer


Output layer
Concat
Hidden 2
Hidden 1
Normalization
Normalization
Input wide
Input deep

Functional API: Nhiều đầu ra (Multiple Outputs)



Hình 10-15: Mạng có nhiều đầu ra

- Tương tự, một mạng cũng có thể có nhiều đầu ra.
- **Ví dụ:** Phát hiện vật thể trên ảnh (1 Output là Phân loại vật, 1 Output khác là Tọa độ đóng khung vật).
- Mỗi Output sẽ có một loss riêng biệt. Keras tính toán theo tổng các loss có hệ số nhân.

Gemini Notebook

6.6 Subclassing API (Lập trình Hướng đối tượng)



Mô hình Tĩnh: Sequential & Functional API	Mô hình Động: Subclassing API
• Dạng Tuyên bố (Declarative). Đồ thị cố định.	• Cấu trúc linh hoạt. Hỗ trợ vòng lặp, if/else trực tiếp trong kiến trúc.

```
class WideAndDeepModel(keras.models.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(units, activation=activation)
        self.hidden2 = keras.layers.Dense(units, activation=kwargs)
        self.main_output = keras.layers.Dense(1)

    def call(self, inputs):
        # Dinh nghia cach truyen tien tai day
        hidden1 = self.hidden1(inputs)
        hidden2 = self.hidden2(hidden1)
        concat = keras.layers.concatenate([inputs, hidden2])
        return self.main_output(concat)
```

Gemini Notebook

7.1 Lưu và Khôi phục Mô hình

Quá trình huấn luyện Mạng Nơ-ron tốn rất nhiều thời gian (Có thể vài ngày). Bạn cần lưu lại!

Định dạng HDF5 (.h5)



- Toàn bộ Trọng số
- Cấu trúc
- Optimizer

Lưu Mô Hình

```
# Luu model
model.save("my_keras_model.h5")
```

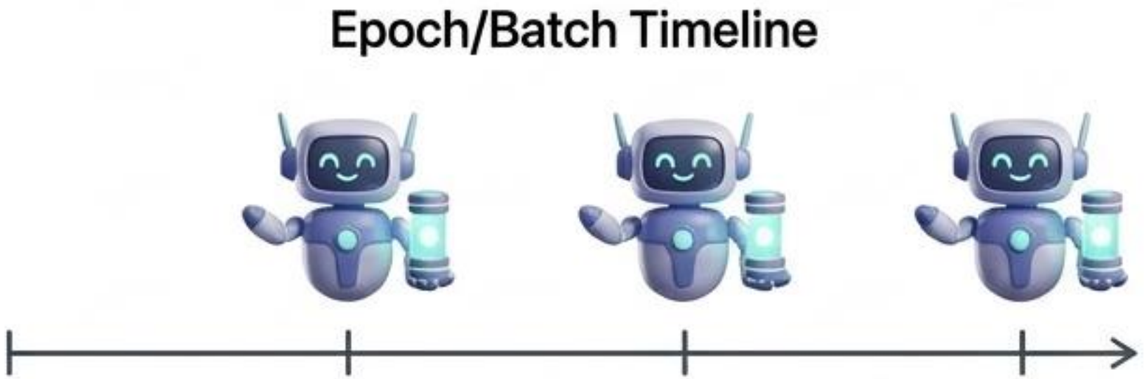
Khôi Phục Mô Hình

```
# Phuc hoi model
model = keras.models.load_model("my_keras_model.h5")
```

Gemini Notebook

7.2 Callbacks: Trợ lý huấn luyện

Keras cho phép gọi các hàm **Calllback ở mỗi cuối Epoch** hoặc cuối **Batch**.



ModelCheckpoint

- Tự động lưu mô hình mỗi khi nó Đạt điểm cao nhất trên tập Validation.
- **Mục đích:** Tránh mất mô hình tốt nhất nếu bị sập nguồn.

```
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    "my_best_model.h5",
    save_best_only=True
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_valid, y_valid),
    callbacks=[checkpoint_cb]
)
```

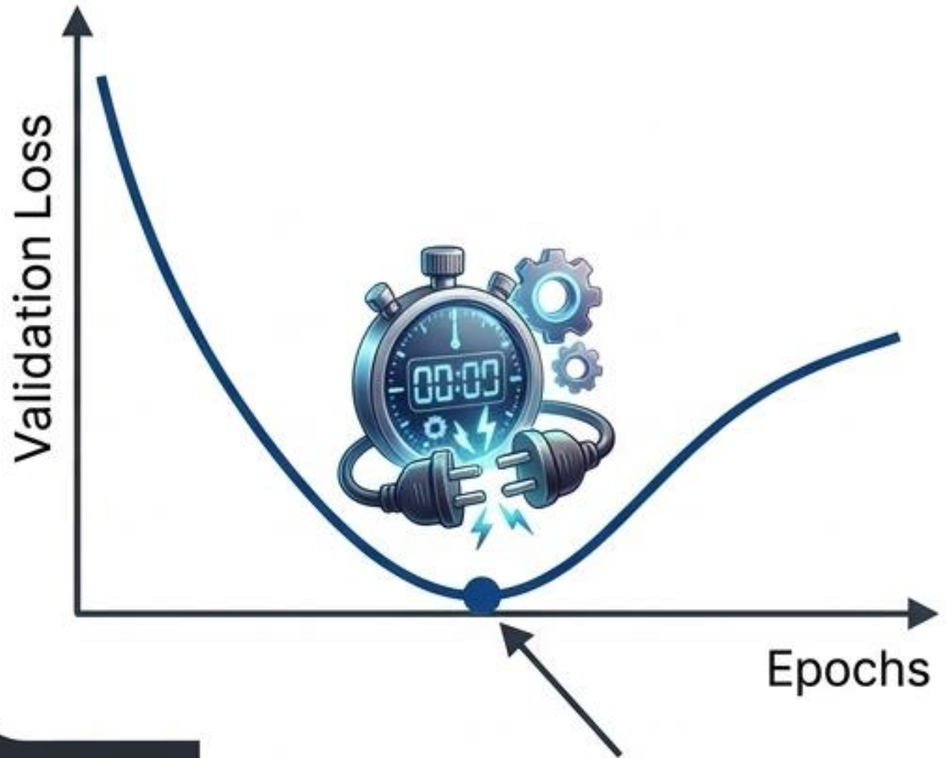
Gemini Notebook

7.3 Dừng sớm (Early Stopping)

Không ai biết cần đặt epochs bao nhiêu là vừa. Đặt lớn quá sẽ gây Overfitting.

```
early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=10,
    restore_best_weights=True
)
# patience=10 nghia la doi them 10 epoch xem co hy vong khong.

history = model.fit(
    X_train, y_train,
    epochs=1000, # Dat epoch 1000 cho thoai mai
    validation_data=(X_valid, y_valid),
    callbacks=[checkpoint_cb, early_stopping_cb]
)
```



Ear'lyStopping: Tự động Rút phích cắm dừng huấn luyện nếu nhận thấy Loss trên tập Validation không còn cải thiện.

Gemini Notebook

7.4 TensorBoard: Công cụ Giám sát tối thượng

Giao diện đồ họa siêu cấp của TensorFlow để theo dõi quá trình huấn luyện trực tiếp.

Real-time



Vẽ đồ thị Loss, Accuracy theo thời gian thực.

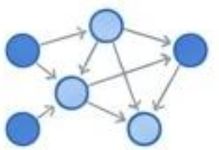
Debugging



Cực kỳ hữu ích để gỡ lỗi các mạng khổng lồ.



Architecture



Hiển thị sơ đồ đồ thị kết nối Lớp (Graph).

Terminal Box

Ghi log bằng cách truyền vào callbacks:

```
keras.callbacks.TensorBoard(log_dir="logs")
```

Gemini Notebook

8.1 Tinh chỉnh Siêu Tham số (Hyperparameters)

Mạng Nơ-ron có độ linh hoạt tuyệt đối -> Hàng tá thông số cần tinh chỉnh! (Ví dụ: 2 lớp hay 100 lớp ẩn?).



Chiến lược Tìm kiếm
Phương pháp Cơ bản: GridSearchCV (Duyệt toàn bộ).
Mạng Sâu (Khuyên dùng): RandomizedSearchCV - Hiệu quả hơn nhiều cho Deep Learning.

Tối ưu hóa Chuyên sâu

Nâng cấp kỹ thuật với công cụ chuyên biệt Keras Tuner.

Gemini Notebook

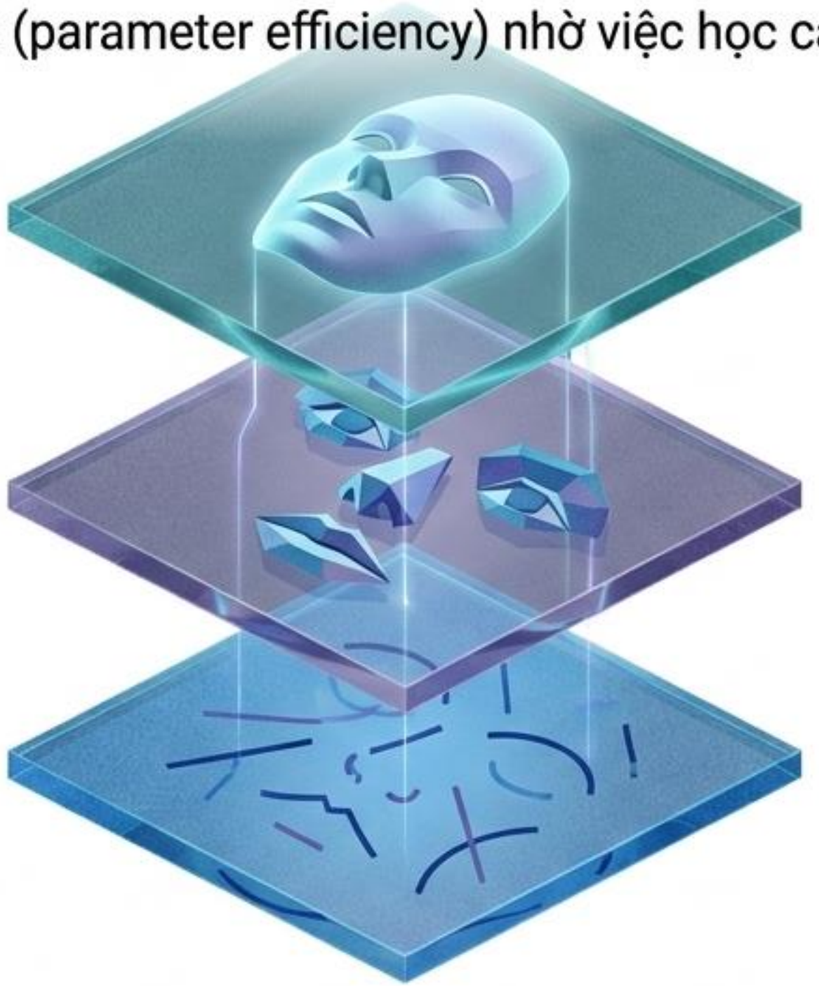
8.2 Số lượng Lớp Ẩn (Hidden Layers)

Mạng 1 lớp ẩn rộng có thể xấp xỉ bất kỳ hàm nào, nhưng rất tốn tham số.
Mạng Sâu có hiệu suất tham số lớn hơn (parameter efficiency) nhờ việc học cách phân cấp dữ liệu.

Lớp 3: Gộp thành khuôn mặt.

Lớp 2: Vẽ thành mắt/mũi/miệng.

Lớp 1: Học vẽ đoạn thẳng.



Visual Hierarchy (Nhận diện mặt)

Kinh nghiệm (Rule of Thumb)

- Bắt đầu với **1 hoặc 2 lớp ẩn** (tốt cho đa số bài toán).
- Ảnh phức tạp: Cần hàng chục lớp.

Gemini Notebook

8.3 Số lượng Nơ-ron trên mỗi Lớp Ẩn

Cấu trúc Cổ điển (Hình phễu)



- **Nơ-ron giảm dần:** 300 -> 100 -> 50.
- **Mục đích:** Tinh lọc dữ liệu.

Cấu trúc Hiện đại (Hình trụ)



- **Đặt số nơ-ron bằng nhau** (VD: tất cả 150).
- **Kết quả:** Hiệu suất tương tự hoặc tốt hơn, giảm bớt cấu hình tham số.

Tactical Tip

Mẹo vặt Chiến thuật

Hãy chọn mô hình có số lớp và số nơ-ron thừa thãi nhiều hơn mức cần thiết, sau đó sử dụng Early Stopping và Dropout để ngăn Overfitting.

Gemini Notebook

8.4 Tốc độ học (Learning Rate)

Tốc độ học (lr) là siêu tham số QUAN TRỌNG NHẤT. Nếu chỉ có thời gian tinh chỉnh 1 tham số, hãy tinh chỉnh cái này!



(Zone: Quá nhỏ)

Mất rất nhiều ngày tháng để huấn luyện.

Giá trị khởi điểm kinh điển: 10^{-3} hoặc 10^{-4}

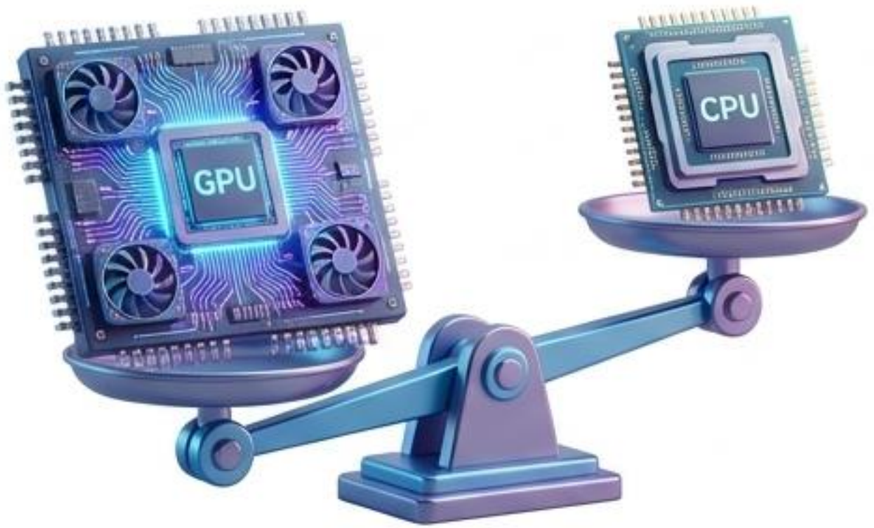
(Zone: Quá lớn)

Mạng nhảy qua lại, không thể hội tụ (Diverge).

Gemini Notebook

8.5 Kích thước Lô (Batch Size) & Optimizer

Kích thước Lô (Batch Size: 32 - 128)



- **Batch Size To:** Huấn luyện nhanh nhờ GPU, nhưng **nguy cơ rơi vào tối ưu cục bộ.**
- **Batch Size Nhỏ:** Tốn CPU, thời gian lâu nhưng mang tính nhảy múa cao, dễ thoát hố sâu cục bộ.

Optimizer (Bộ tối ưu)



- **Thuật toán Adam** (Sẽ học ở chương 11) được chứng minh tốt hơn nhiều so với SGD cổ điển.
- **Thực tiễn:** Đang được dùng mặc định ở hầu hết bài toán Học sâu.

Gemini Notebook



HỎI & ĐÁP

Gemini Notebook